



Detecting Stack Overflows in RTOS-Based Designs - Part 2

This article is written by Jean J. Labrosse, RTOS Expert.

There are many techniques that can be used to detect stack overflows. Some make use of hardware while some are performed entirely in software. As we will see shortly, having the capability in hardware is **by far** preferable since stack overflows can be detected immediately as they happen and in fact, avoided since a write to an invalid access is actually prevented by the hardware.

Hardware stack overflow detection mechanisms generally triggers an exception handler. The exception handler typically saves the current PC (Program Counter) and possibly other CPU registers onto the current task's stack.

Of course, because the exception occurs when we are attempting to access data outside of the stack, the handler would overwrite some variables or another stack in your application; assuming there is RAM beyond the base of the overflowed stack.

In most cases the application developer will need to decide what to do about a stack overflow condition. Should the exception handler place the embedded system in a known safe state and reset the CPU or simply do nothing?

If you decide to reset the CPU, you might figure out a way to store the fact that an overflow occurred and, which task caused the overflow so you can notify a user upon reset.

Technique 1: Using a Stack Limit Register

Some processors (unfortunately very few of them) have simple yet highly effective stack pointer overflow detection registers. This feature is available on processors based on the ARMv8-M CPU architecture. When the CPU's stack pointer goes below (or above depending on stack growth) the value set in this register (let's call it the

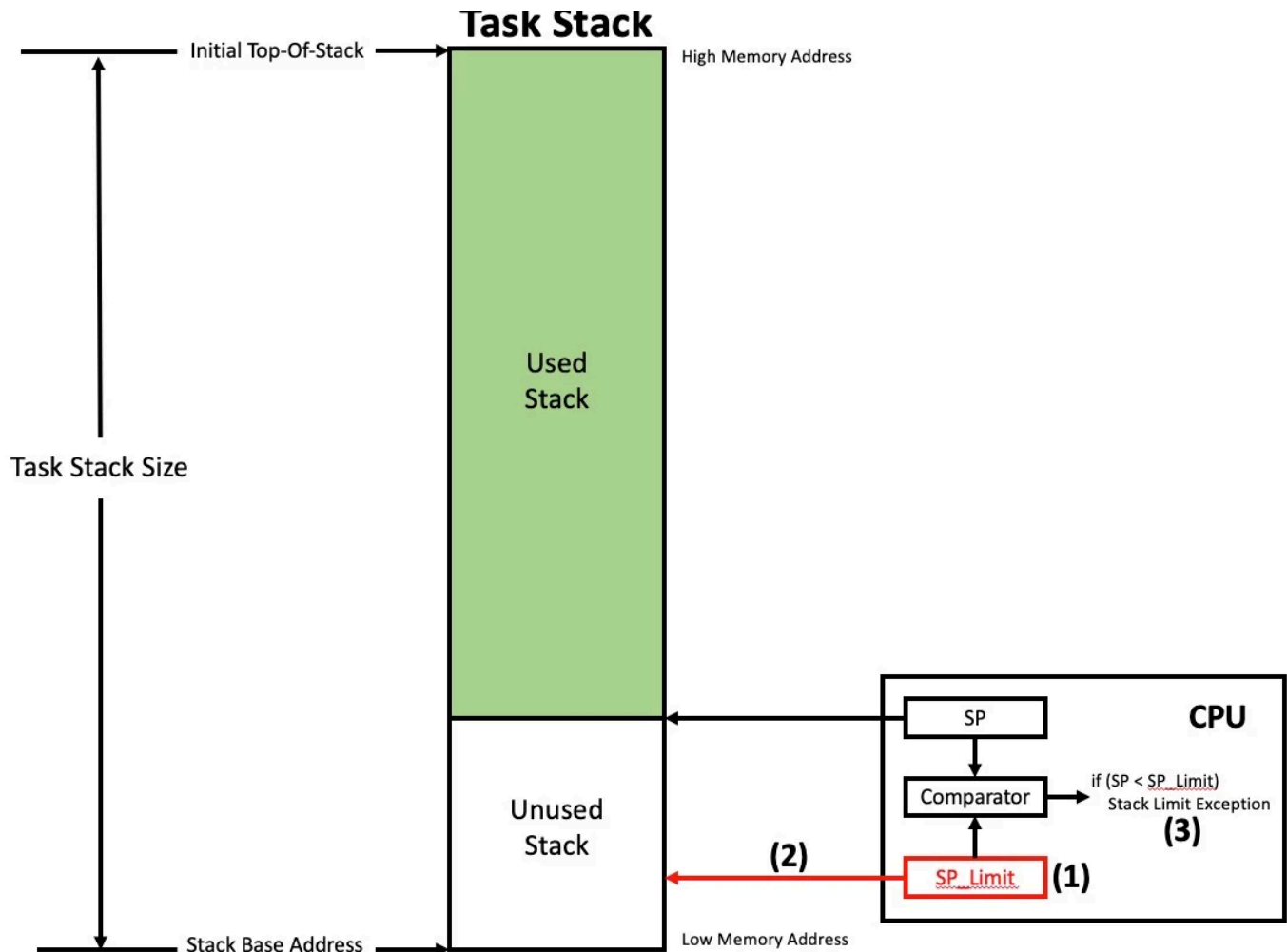


Figure 3 – Using a Stack Limit Register to Detect Stack Overflows

- (1) The **SP_Limit** register is loaded by the context switch code of the RTOS when the task is switched in.
- (2) The location where the **SP_Limit** points to could be at the very base of the stack or, preferably, at a location that would allow the exception handler enough room to save enough registers on the offending stack to handle the exception.
- (3) As the stack grows, if the **SP** register ever goes below the **SP_Limit**, an exception is generated. The attempt to change the **SP** register would be denied thus keeping the application code within a valid stack.

As a side note, μ C/OS-III and Cesium/OS3 were designed from the get-go to support CPUs with a stack limit register although at the time, there was only one processor that supported that feature, the Infineon 80C166/167.



upgraded, most likely by the RTOS developer.

The value of the SP_Limit register used by the CPU's stack overflow detection hardware needs to be changed whenever the RTOS performs a context switch. The sequence of events to do this **must be** performed in the following order:

- 1) Set SP_Limit to 0. This ensures the SP register is never below the SP_Limit. Note that I assumed here that the stack grows from high-memory to low-memory but the concept works in a similar fashion if the stack grows in the opposite direction.
- 2) Load the SP
- 3) Get the value of the SP_Limit that belongs to the new task from its TCB. Set the SP_Limit register to this value.

Technique 2: Using an MPU

Many of the current processors are equipped with a Memory Protection Unit (MPU) which typically monitors the address bus to see if your code is allowed to access certain memory locations or I/O ports. MPUs are relatively simple devices to use but are somewhat complex to setup.

However, if all you want to do is detect stack overflows then an MPU can be put to good use without a great deal of initialization code. If the MPU is already on your chip, meaning it's available at no extra cost to you, why not use it? In the discussion that follows, we'll setup an MPU region that says: "if ever you write *outside* this stack area (i.e. region), the MPU will generate an exception".

Of course, if you have RAM in other locations, you would setup another MPU region to provide permission to access that RAM. For sake of discussion, I'll assume you are using a Cortex-M from the ARMv7M architecture and those are typically equipped with an 8-region MPU.

In this case, we can put an MPU region *around* the *whole* stack. On the ARMv7M, the stack size **must be** a power-of-two (minimum 32 bytes) and **align** on the same power-of-two base address. In other words, the stack must be either 32, 64, 128, 256, 512, ... bytes in size and align on 0x?????E0, 0x?????C0, 0x?????80, 0x?????00, 0x?????E00) ..., respectively.

On processors based on the ARMv8-M architecture, this restriction has been removed and an MPU region size granularity is 32 bytes and the size can be any



One way to setup your stacks is to locate **ALL** of the stacks together in contiguous memory, starting the stacks at the base of RAM as shown in Figure 4.

How you do this is beyond the scope of this paper but, the IAR Linker makes it easy.

Task Stacks

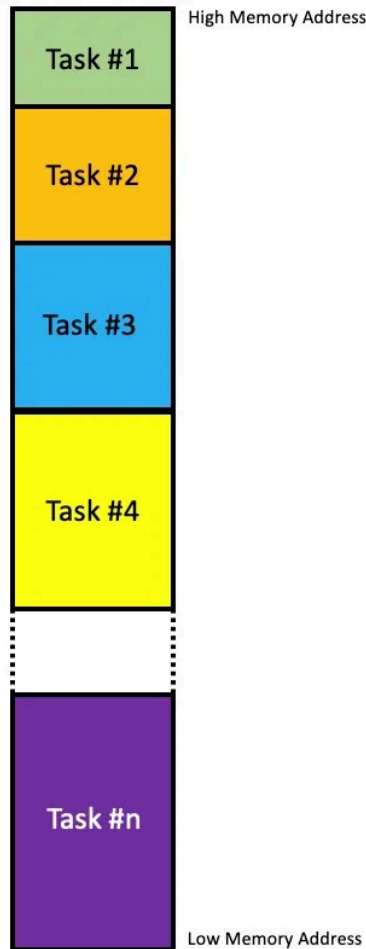


Figure 4 – Locating Task Stacks Continuously

As the RTOS context switches between tasks, it moves a single MPU ‘protection window’ as shown in Figure 5. In other words, when a task switch occurs, the RTOS reprograms the MPU to put one of its regions around the new task’s stack (Red Box).

On an ARMv7M (and ARMv8M), it's preferable to use the *last region* (i.e. region #7 on an 8 region MPU) because this region supersedes the other regions (assuming you are using the other regions of the MPU).

One of the restrictions with this method is that you cannot allocate buffers on a task’s stack and pass a pointer to that buffer to another task because the other task would



slapped by an MPU violation is a kind punishment.

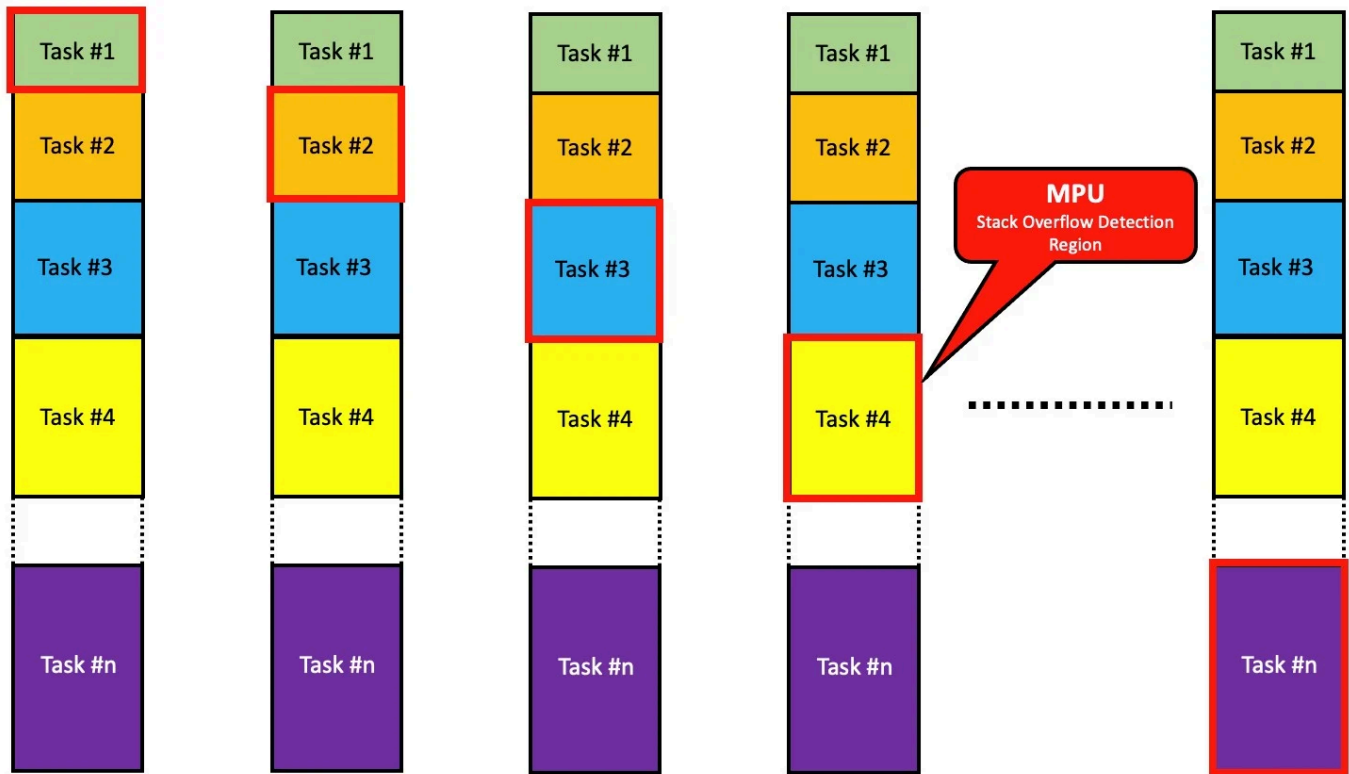


Figure 5 – Moving the MPU Region During Context Switches

Technique 3: Hardware-based RED-Zones

Another way to use the MPU is to place a region *outside* the stack area of the task that is running. If a write occurs *inside* the MPU region the MPU triggers an exception.

Note that in this case the MPU is configured differently than the previous method, instead of generating an exception when your code writes *outside* the region, the violation occurs when your code writes *inside* the region as shown in Figure 6.

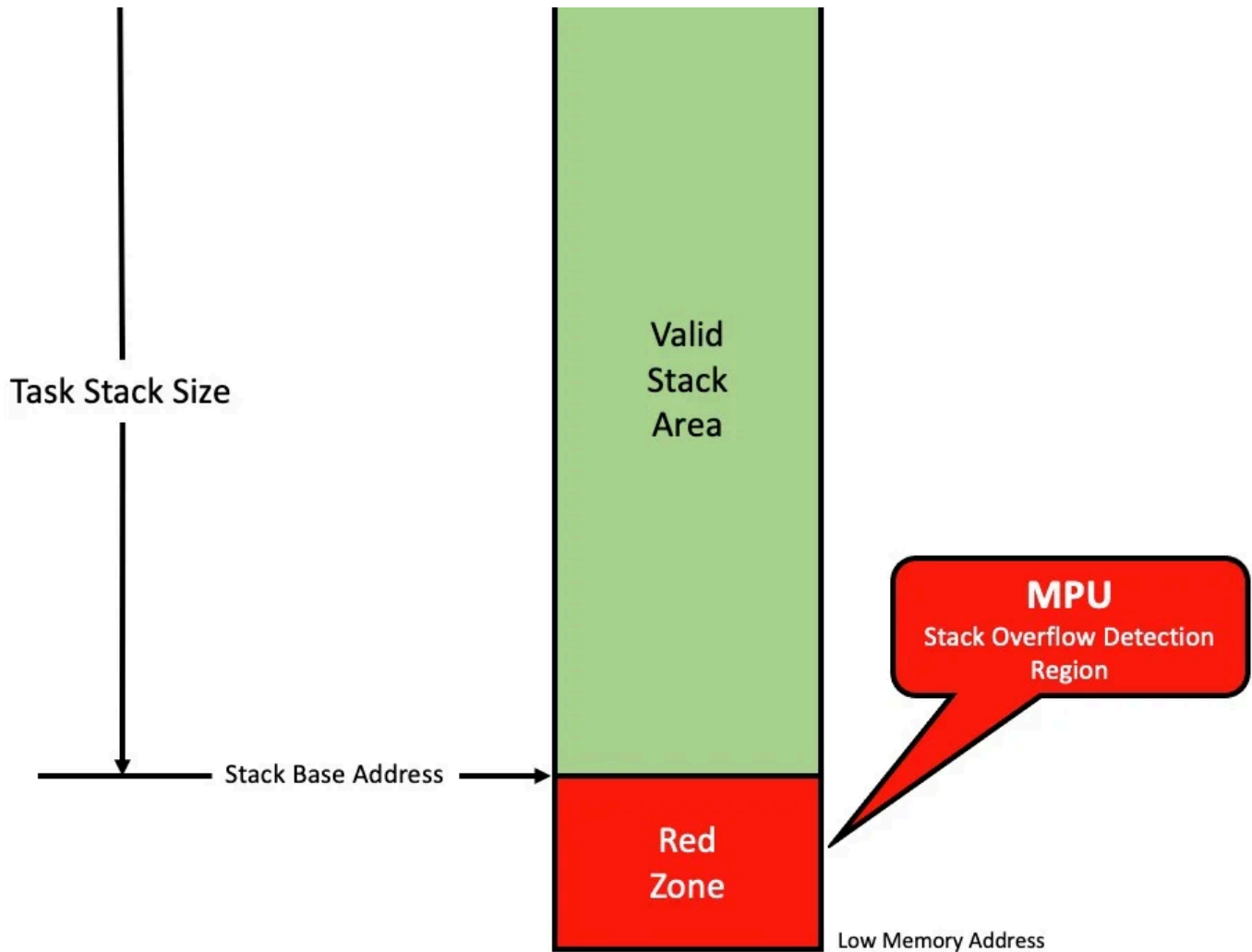


Figure 6 – Using an MPU region to catch writes outside the stack area

This method is not as reliable as the previous one unless you make the region really large in order to catch allocation of large arrays or structures on a task stack.

The other problem is that you need make sure you are not placing the red zone to overlap memory that needs to be accessible to the task.

The biggest advantage is that you don't need to set the stack size of a task to be a power-of-two. In most cases, 128 to 256 bytes of red zone should be sufficient.

Technique 4: Software-based RED-Zones

If you don't have or don't want to use the MPU then you might consider a software-based RED-Zone. However, this method needs to be supported by the RTOS you are



stack as shown in Figure 7.

The size of the red-zone depends on your appetite for risk, overhead and the amount of RAM you want to dedicate on each stack for this red-zone. During task initialization, the RTOS places a known pattern, something like 0xABCDEF01 (but could be almost anything else) in the RED-Zone.

During a context switch, the RTOS checks to see if any of the RED-Zone locations have been changed and if so, the RTOS calls a function to take corrective actions.

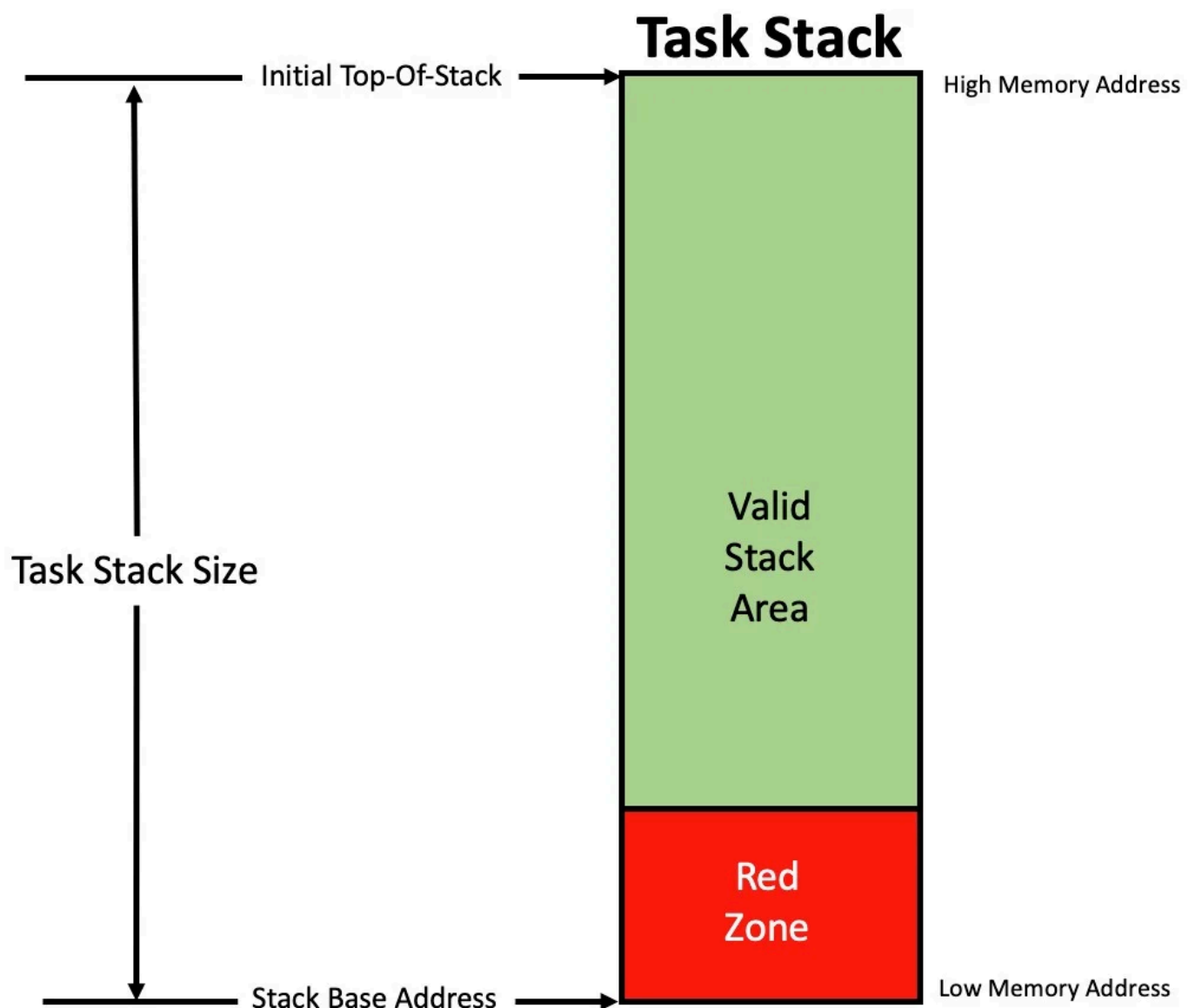


Figure 7 – Red-zone is checked by the RTOS on a context switch



If the usable area of your task stack needs to be 512 bytes and your RED-Zones are all 128 bytes then you'd need to allocate 640 bytes and thus your allocated RAM only has 80% efficiency!

If you make the RED-Zone small, it's ever so important to initialize local variables, large arrays or data structures upon entry of a function in order to detect an overflow using this method.

The software RED-Zone is nice because it's portable across any CPU architecture. However, the drawback is that it consumes possibly valuable CPU cycles during a context switch as well as RAM.

Technique 5: Examining the SP register during a context switch

A technique, used by FreeRTOS, is to examine the value of the stack pointer during a context switch. If the value is less than the base address of the stack then, the RTOS would know the task wrote something outside its stack.

This is certainly the worst of all the methods because your code needs to be preempted at the exact moment when the stack pointer is outside the stack! This could be way too late and untold damage could occur.

However, if you use a software-based stack limit and test against that instead of the stack base address then, you'd have a better chance to catch the stack pointer before the task can do any damage.

Want to learn more?

Check out [Part 1, Stack Overflows in RTOS-Based Designs](#), or access the on-demand webinar, [Tips and hints for better debugging your RTOS-based application](#).

About the author



You might also be interested in:





World leader of software and services for embedded systems development.

Products

[Overview](#)

[Architectures](#)

[Industries](#)

[Requirements](#)

Investors

[About IAR Systems Group](#)

[Corporate governance](#)

[Reports and news](#)

About

[News](#)

[Career](#)

[Contact Us](#)

[Privacy policy](#)

[Cookies](#)

[Trademarks](#)

[Terms of Use](#)

[Code of Conduct](#)

[Vulnerability Disclosure](#)